

Ranking and Unranking of t -ary Trees Using the Right Distance Representation

Ro-Yu Wu¹ Jou-Ming Chang² Yue-Li Wang^{3,*}

¹ Department of Industrial Engineering Management, Lunghwa University of Science and Technology, Taoyuan, Taiwan, ROC

² Department of Information Management, National Taipei College of Business, Taipei, Taiwan, ROC

³ Department of Computer Science and Information Engineering, National Chi Nan University, Nantou, Taiwan, ROC

Abstract

In this paper, we introduce a concise representation, called right-distance sequences (or RD-sequences for short), to describe all t -ary trees with n internal nodes. A result reveals that there exists a close relationship between the representation and the well-formatted integer sequences suggested by Zaks in 1980. Using recursion tree and its concomitant tables, a systematical way can help us to investigate the structural representation of t -ary trees. Consequently, we develop efficient algorithms for determining the rank of a given t -ary tree (i.e., ranking algorithm), and for converting a positive integer to its corresponding RD-sequence (i.e., unranking algorithm). Both the ranking and unranking algorithms can be run in $O(tn)$ time and without really building any auxiliary table.

Keywords: t -ary trees; Ranking; Unranking; Lexicographic order; Recursion tree;

1. Introduction

Many algorithms have been developed for generating all t -ary trees (as well as binary trees) with n internal nodes. In most of these algorithms, a t -ary tree is encoded to an integer sequence and the sequences of different t -ary trees are generated in a particular order, especially in lexicographic order [1, 2, 4, 10, 12, 22, 23, 25–27, 29, 32, 33] or Gray-code order (i.e., any two successive sequences are different in only one element at the same position) [3, 14, 21, 30]. Besides, a few algorithms employ the usual pointer in computer representation to generate t -ary trees [15, 31]. For simplicity, we sometimes say t -ary trees to mean their representations.

Typically, an important issue for algorithms to enumerate various classes of objects is that it must intend to run in a constant time for each generation. Consequently, a number of algorithms have been proposed to generate t -ary trees with constant amortized time (i.e.,

*All correspondence should be addressed to Professor Yue-Li Wang, Department of Computer Science and Information Engineering, National Chi Nan University, University Rd. Puli, Nantou Hsien, Taiwan 545 Republic of China (Email: ylwang@cs.ntust.edu.tw).

constant average time per tree) [1, 5, 13, 22, 23, 25, 28, 31, 32]. Moreover, the so-called loopless algorithms (i.e., algorithms involve no recursion and, after initialization, only uses “if-then-else” statements and assignment statements) with worst-case constant time per tree have attracted a great deal of attention in the recent literature [12, 14, 15, 21, 29–31]. For parallel generation of t -ary trees, the cost-optimal algorithms with constant delay can be found in [4, 27] for linear array model, [26] and [2] for CREW/EREW shared memory model, and [10] for associative processor model.

Given a specific order on the set of t -ary trees with n internal nodes, the *ranking algorithm* is a function that determines the position (rank) of a given t -ary tree in that order, and the *unranking algorithm* is one that finds the tree of a given position. Ranking and unranking algorithms of t -ary trees have appeared in [1, 5, 11, 22, 23, 25, 33]. Till now, both the best known sequential ranking algorithms [1, 22, 23, 25, 33] and unranking algorithms [1, 11, 23] have complexity in $O(tn)$ time. Contrastively, to the best of our knowledge, Kokosiński’s algorithm [11] is the only one for unranking t -ary tree in parallel. Such an algorithm can be run in $O(n)$ time and relies on a parallel architecture with tn processors that furnishes with systolic and associative features. Noteworthily, there is a controversy in the evaluation of complexity analysis for all the above mentioned algorithms. All of them are required to build a coefficient table of size approximating to $(n(t-1)+1) \cdot n$ in advance before ranking or unranking. As a result, the pre-computation of those coefficient entries takes additional cost in $O(tn^2)$ time for all the proposed sequential algorithms and in $O(tn)$ time for Kokosiński’s parallel algorithm.

A challenge arisen from this research is to ask whether the pre-computation for building coefficient table is necessary in order to compute the ranking and/or unranking function.

Lucas et al. claimed that, in fact, they have been unable to reduce the running time of the ranking or unranking algorithms to less than $O(n^2)$ time even if they consider for binary trees (see p. 354 in [17]). In this paper, we deal with the problem of ranking and unranking for t -ary trees. We will show that both the ranking and unranking problems for t -ary trees can be solved in $O(tn)$ time and without really building the coefficient table. Indeed, not all the entries of the table are necessary to compute for ranking or unranking. An observation shows that all entries needed for computation are located in a path from a lower-left corner to an upper-right corner of the table. Thus, the idea behind our algorithms is that only the entries along the path we require to compute. Then, we establish four formulas that could be used for computing the relative right, left, upper, and lower entries of a given element in the table, respectively. Because each of the formulas can be computed in $O(1)$ time and the length of path is no more than tn , all subsequent entries can be determined quickly if an initial term is computed. Consequently, we obtain an improvement on the complexity of time and space for both the ranking and unranking algorithms.

2. Structural representations of t -ary trees

We begin with a structure representation for t -ary trees. For $t \geq 2$, a t -ary tree T is an ordered tree defined recursively as follows: either T is empty or it has a distinguished node r called its *root* that is connected to subtrees $T_1, T_2, \dots, T_t$, each of which is a t -ary tree. Let $\mathcal{T}_{n,t}$ denote the set of t -ary trees with n internal nodes. If $T \in \mathcal{T}_{n,t}$, then we assume that all internal nodes of T are numbered from 1 to n , according to the preorder traversal of T (i.e. visit recursively the root and then the subtrees of T from left to right). For each internal node $i \in T$, we define d_i as follows. First, $d_i = 0$ for each node i on the

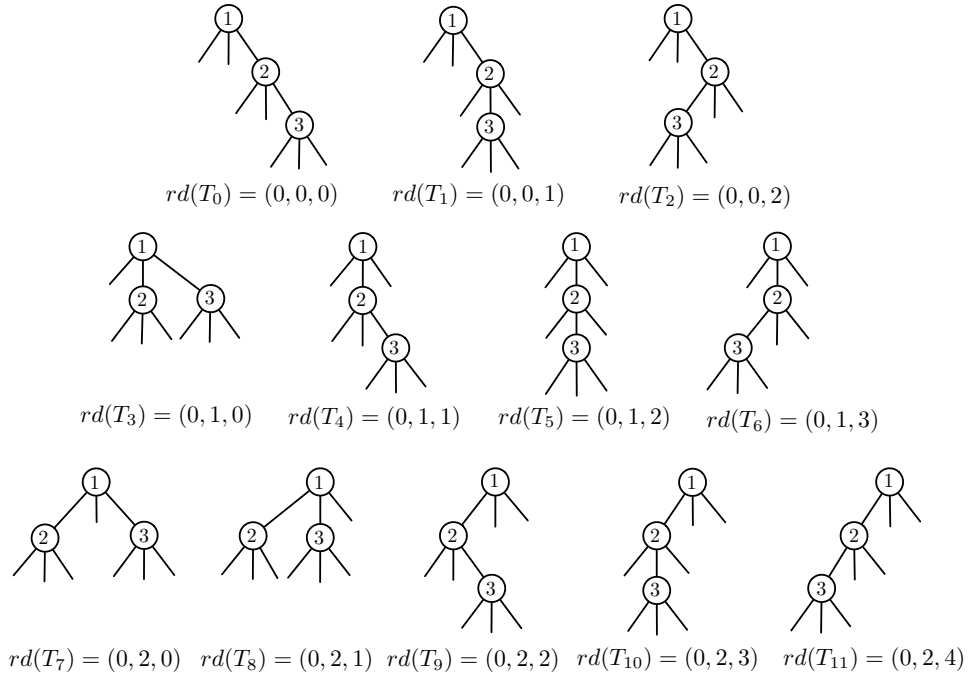


Figure 1: The 3-ary trees with 3 internal nodes.

right arm of T (i.e., the path from the root to its rightmost node). For all other nodes, $d_i = d_{p(i)} + t - k$ ($1 \leq k \leq t$), where $p(i)$ stands for the parent of i and i is the k th child (from left to right) of $p(i)$. Thus, $d_i = d_{p(i)}$ when i is the rightmost child of its parent. Note that $d_i \leq d_{i-1} + (t - 1)$. Denote that resulting sequence by $rd(T) = (d_1, d_2, \dots, d_n)$ as the *right distance sequence* (RD-sequence for short) of T .

A synonym of RD-sequence is called the *inversion table* due to [20] and [4]. In this paper, we use the more apposite term “RD-sequence” to indicate the measurement of distance that was introduced by Mäkinen [18, 19] for binary trees. However, Mäkinen’s distance representation uses the left arm (i.e., the path from the root to its leftmost node) of the tree to measure the distance of a node, and thus is referred to as the *left distance method*. Recall from [8, 9] that the number of t -ary trees is given by $|\mathcal{T}_{n,t}| = \frac{1}{tn+1} \binom{tn+1}{n} = \frac{1}{(t-1)n+1} \binom{tn}{n}$. Figure 1 depicts twelve 3-ary trees for $n = 3$ and their RD-sequences.

In [32], Zaks defined another representation, called *Z-sequence*, of a t -ary tree T as follows. For each internal node $i \in T$, denote by z_i the visited order of i in the pre-order traversal of T , where the count of visited nodes includes internal nodes and external nodes in the traversal. The resulting sequence $z(T) = (z_1, z_2, \dots, z_n)$ is called the Z-sequence of T . Note that every Z-sequence satisfies $z_{i-1} < z_i \leq 1 + t(i - 1)$. For example, Figure 2 shows the RD-sequence and the Z-sequence of a particular 4-ary tree T with 6 internal nodes.

The following theorem reveals close relationship between the representations of RD-sequences and Z-sequences. For more complete discussion about the relationship among various representations can refer to [17].

Theorem 1 *Let T be a t -ary tree. Then $d_i + z_i = 1 + t(i - 1)$ for every internal node $i \in T$.*

Proof. Let K be the set of external nodes visited between two internal nodes i and $i + 1$

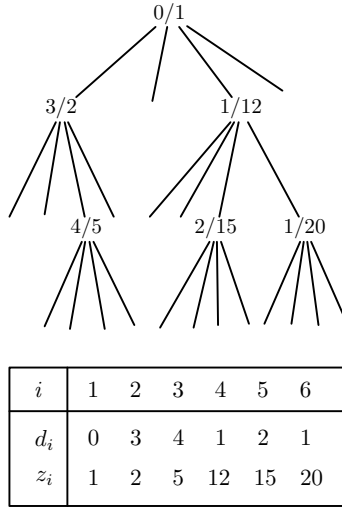


Figure 2: A 4-ary tree T such that every internal node $i \in T$ is labeled by d_i/z_i .

in the preorder traversal of T and $k = |K|$. By definition, we have the following relevance:

$$z_{i+1} = z_i + (k + 1) \quad (1)$$

We now consider the following two cases to show the relevance between d_i and d_{i+1} using the term k . The first case we consider when $0 \leq k < t$. In this case, $i + 1$ is the $(k + 1)$ th child of i , and thus $d_{i+1} = d_i + t - (k + 1)$. The next case we consider when $k \geq t$. We denote by $c(x) = l$ to mean that the node x is the l th child of its parent. Also, let lca_i denote the least common ascendent of i and $i + 1$ in T . Obviously, all the t children of i are external nodes and lca_i is the parent of $i + 1$. Suppose that $c(i + 1) = s$ and the unique path from lca_i to i is $lca_i = x_0, x_1, \dots, x_p = i$. Then, for each $j = 1, \dots, p$, the right distance $d(x_j)$ of node x_j can be computed by $d(x_j) = d(x_{j-1}) + t - c(x_j)$. By iterative substitution, we have

$$d(x_p) = d(x_0) + p \cdot t - \sum_{j=1}^p c(x_j)$$

Since $d_{i+1} = d(x_0) + t - s$ and the number of nodes in K which are not the children of i is $k - t = (s - 1) - c(x_1) + \sum_{j=2}^p (t - c(x_j))$, this

comes into $d_i = d(x_p) = d_{i+1} - t + (k + 1)$. Therefore, we obtain the following relevance from the above argument:

$$d_{i+1} = d_i + t - (k + 1) \quad (2)$$

From (1) and (2) we obtain

$$d_{i+1} + z_{i+1} = d_i + z_i + t.$$

Since $d_1 = 0$ and $z_1 = 1$, we can solve the recurrence (i.e., an arithmetic sequence $d_i + z_i$ with the given initial term $d_1 + z_1 = 1$) to obtain the desired result. $\square$

According to Theorem 1, it follows that to generate t -ary trees on n nodes using RD-sequences in lexicographic order, it suffices to generate t -ary trees using Z-sequences in the reverse of lexicographic order, and vice versa.

In the following, we apply a systematical way to describe all t -ary trees in $\mathcal{T}_{n,t}$. Given integer $t \geq 2$, the t -ary recursion tree with n levels is a rooted tree $\mathbb{T}$ constructed by the following rules: (1) Every node in $\mathbb{T}$ is associated with a labeling; (2) The first level contains a single node (i.e., the root) with labeling 0; (3) Every nonleaf node (i.e., node not in the n th level) with labeling k has $t + k$ children and such that these children are labeled by $0, 1, \dots, t + k - 1$ from left to right in the next level of $\mathbb{T}$. For simplicity, we merely mention *recursion tree* if t and n are inessential. A result will be shown in Corollary 3 that the i th level of recursion tree totally contains $|\mathcal{T}_{i,t}|$ nodes. Consequently, the full labeling along the path from the root to a leaf in $\mathbb{T}$ represents an RD-sequence of a t -ary tree with n internal nodes. Due to the relationship of d_i and z_i stated in Theorem 1, there is another way to label the nodes of recursion tree. Figure 3 demonstrates the 3-ary recursion tree with three levels. In addition, it also shows the two types of sequences for 3-ary trees and their corresponding ranks.

The similar concept of recursion tree was independently introduced under various

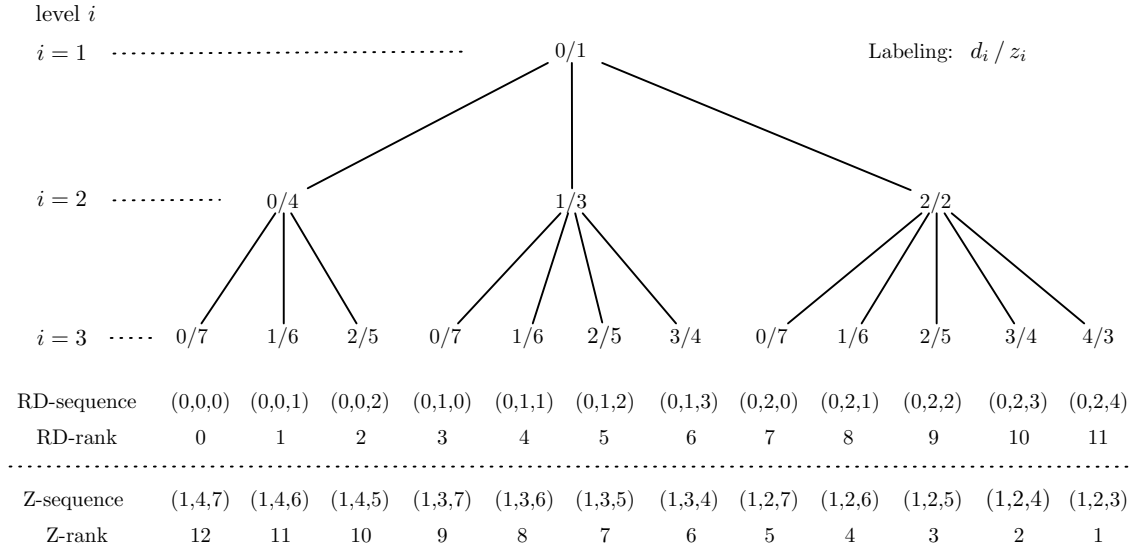


Figure 3: The recursion tree, RD-sequences, Z-sequences, and the rank of 3-ary trees.

names including backtracking tree [34] and recursion tree [17] for $t = 2$ (i.e., the case for representing binary trees), and coding tree [28] for $t \geq 2$. However, all homologues can be obtained from the recursion tree constructed above by flipping the shape in horizontal. For thoroughly studying the structure of t -ary recursion tree, we first observe the following facts.

Fact 1 *The degree (i.e., the number of children) of a node x in the i th level of $\mathbb{T}$ is at most $(t-1)i + 1$.*

Fact 2 *For every two nodes x and y with the same degree in the same level of $\mathbb{T}$, the subtree rooted at x and the subtree rooted at y are isomorphic.*

For each level $i = 1, \dots, n$ and $0 \leq k \leq (t-1)(i-1)$, let $X_{i,k}$ be the set consisting of nodes with labeling $d_i = k$ in the i th level of $\mathbb{T}$. Note that the range of k is determined from Fact 1. It is clear that every node in $X_{i,k}$ has the same degree. According to Fact 2, we may denote by $A_{i,(t-1)(n-i)+k}$ the number of leaves in the subtree of $\mathbb{T}$ rooted at a node $x \in X_{i,k}$. Obviously, $A_{n,k} = 1$ for $0 \leq k \leq (t-1)(n-1)$ (i.e., the case of $i = n$ and thus every subtree

rooted at a leaf of $\mathbb{T}$ contains singleton). Similarly, we have $A_{n-1,(t-1)+k} = t+k$ for $0 \leq k \leq (t-1)(n-2)$ (i.e., the case of $i = n-1$). In general, for counting the number of leaves in the subtree of $\mathbb{T}$ rooted at a node x in the level $i \leq n-1$, by the decomposition of tree structure we may separately count the number of leaves in the subtree rooted at each child of x , and then sum up these counts for total. Thus we have the following formula:

$$A_{i,(t-1)(n-i)+k} = \sum_{j=(t-1)(n-i-1)}^{(t-1)(n-i)+k} A_{i+1,j} \quad (3)$$

where $0 \leq k \leq (t-1)(i-1)$. Using (3) we can prove the following by induction (see Appendix A).

Theorem 2 *Given integers $t \geq 2$ and $n \geq 1$, for each $i = 1, \dots, n$ and $0 \leq k \leq (t-1)(i-1)$ we have*

$$A_{i,(t-1)(n-i)+k} = \frac{m(t+k)}{(mt+k+1)(mt+k)} \binom{mt+k+1}{m}. \quad (4)$$

where $m = n - i + 1$.

Corollary 3 *The i th level of t -ary recursion tree $\mathbb{T}$ totally contains $|\mathcal{T}_{i,t}|$ nodes.*

Proof. It can be derived by setting $i = 1$ and substituting n for i in (4). Since $k = 0$ and

$m = i$, this yields

$$A_{1,(t-1)(i-1)} = \frac{1}{it+1} \binom{it+1}{i} = |\mathcal{T}_{i,t}|. \quad \square$$

Using (4) we can build a table called the t -ary recursion table. The special case when $t = 2$ is the well-known Catalan's triangle table [24]. Table 1 shows the 3-ary recursion table for $n = 6$. This table can be visualized as a staircase. The corner entry of each step is obtain from (4) by taking $k = 0$ and is equal to $A_{i,(t-1)(n-i)} = |\mathcal{T}_{n-i+1,t}|$. A point of view from (3) shows that the value of any entry, excepting those in the bottom row, is equal to the prefix sum of the entry below in the next row (e.g. $A_{3,6} = 55 = \sum_{j=4}^6 A_{4,j} = 12 + 18 + 25$ in Table 1). Thus, the value of each entry, excepting those in the bottom row and corner entries, is equal to the one below plus the one to the left (e.g. $A_{3,7} = 88 = A_{4,7} + A_{3,6} = 33 + 55$ in Table 1). This immediately implies the following.

Corollary 4 *Given integers $t \geq 2$ and $n \geq 2$, for each $i = 1, \dots, n-1$ and $1 \leq k \leq (t-1)(i-1)$ we have*

$$A_{i,(t-1)(n-i)+k} = A_{i,(t-1)(n-i)+k-1} + A_{i+1,(t-1)(n-i)+k} \quad (5)$$

For the efficiency of computation, we also define the following formula:

$$B_{i,(t-1)(n-i)+k} = \begin{cases} 0 & \text{if } k = 0 \\ \sum_{j=0}^{k-1} A_{i,(t-1)(n-i)+j} & \text{otherwise.} \end{cases} \quad (6)$$

where $0 \leq k \leq (t-1)(i-1)$. It means that the term $B_{i,(t-1)(n-i)+k}$ is the prefix sum of the first k nonblank entries in the i th row of recursion table. Hence, a table built from (6) is referred to as t -ary accumulation table. Table 2 shows the case for $t = 3$ and $n = 6$. Using (4) we can rewrite (6) as follows (see Appendix B for proof).

Table 1: The 3-ary recursion table for $n = 6$

A			$(t-1)(n-i)+k$										
n	i	$0 \leq k \leq (t-1)(i-1)$	0	1	2	3	4	5	6	7	8	9	10
6	1	$k = 0$											1428
	2	$0 \leq k \leq 2$									273	455	700
	3	$0 \leq k \leq 4$							55	88	130	182	245
	4	$0 \leq k \leq 6$					12	18	25	33	42	52	63
	5	$0 \leq k \leq 8$			3	4	5	6	7	8	9	10	11
	6	$0 \leq k \leq 10$	1	1	1	1	1	1	1	1	1	1	1

Table 2: The 3-ary accumulation table for $n = 6$

B			$(t-1)(n-i)+k$										
n	i	$0 \leq k \leq (t-1)(i-1)$	0	1	2	3	4	5	6	7	8	9	10
6	1	$k = 0$											0
	2	$0 \leq k \leq 2$									0	273	728
	3	$0 \leq k \leq 4$							0	55	143	273	455
	4	$0 \leq k \leq 6$					0	12	30	55	88	130	182
	5	$0 \leq k \leq 8$			0	3	7	12	18	25	33	42	52
	6	$0 \leq k \leq 10$	0	1	2	3	4	5	6	7	8	9	10

Theorem 5 *Given integers $t \geq 2$ and $n \geq 1$, for each $i = 1, \dots, n$ and $0 \leq k \leq (t-1)(i-1)$ we have*

$$B_{i,(t-1)(n-i)+k} = \frac{k}{mt+k} \binom{mt+k}{m} \quad (7)$$

where $m = n - i + 1$.

In addition, from (7) we can obtain the following corollaries which are useful for designing ranking and unranking algorithms in Section 3.

Corollary 6 *Given integers $t \geq 2$ and $n \geq 2$, for each $i = 2, \dots, n$ and $1 \leq k \leq (t-1)(i-1) - 1$ we have*

$$\begin{aligned} & B_{i,(t-1)(n-i)+(k+1)} \\ &= B_{i,(t-1)(n-i)+k} \cdot \frac{(k+1)(mt+k)}{k[mt(t-1)+k+1]} \end{aligned} \quad (8)$$

where $m = n - i + 1$.

Corollary 7 *Given integers $t \geq 2$ and $n \geq 2$, for each $i = 2, \dots, n$ and $1 \leq k \leq (t-1)(i-1)$ we have*

$$\begin{aligned} & B_{i,(t-1)(n-i)+(k-1)} \\ &= B_{i,(t-1)(n-i)+k} \cdot \frac{(k-1)[mt(t-1)+k]}{k(mt+k-1)} \end{aligned} \quad (9)$$

where $m = n - i + 1$.

Corollary 8 *Given integers $t \geq 2$ and $n \geq 2$, for each $i = 2, \dots, n$ and $t-1 \leq k \leq (t-1)(i-1)$ we have*

$$\begin{aligned} & B_{i-1,(t-1)(n-i)+k} \\ &= B_{i,(t-1)(n-i)+k} \cdot \frac{(k-t+1)(mt+k)}{k(m+1)} \end{aligned} \quad (10)$$

where $m = n - i + 1$.

Corollary 9 *Given integers $t \geq 2$ and $n \geq 2$, for each $i = 2, \dots, n-1$ and $1 \leq k \leq (t-1)(i-1)$ we have*

$$\begin{aligned} & B_{i+1,(t-1)(n-i)+k} \\ &= B_{i,(t-1)(n-i)+k} \cdot \frac{m(k+t-1)}{k(mt+k-1)} \end{aligned} \quad (11)$$

where $m = n - i + 1$.

3. Ranking and unranking algorithms

In this section we first determine the rank of a t -ary tree T according to the RD-sequences in lexicographic order. Let $\mathbb{T}$ be the t -ary recursion tree. Recall that $A_{i,(t-1)(n-i)+k}$ denotes the number of leaves in the subtree rooted at a node with labeling $d_i = k$ in the i th level of $\mathbb{T}$. Moreover, we know from (6) that $B_{i,(t-1)(n-i)+k}$ indicates the total number of leaves for those subtrees rooted at nodes with labels from 0 to $k-1$. Since the full labeling $(d_1, d_2, \dots, d_n)$ along the path from the root of $\mathbb{T}$ to a leaf represents a certain t -ary tree T , we can define the rank of T by using the order (from left to right) of this appointed leaf in $\mathbb{T}$. It follows that

$$\begin{aligned} & \text{Rank}(d_1, d_2, \dots, d_n) \\ &= \sum_{i=1}^n B_{i,(t-1)(n-i)+d_i} \end{aligned} \quad (12)$$

Note that $\text{Rank}(0, 0, \dots, 0) = 0$ and $\text{Rank}(0, t-1, 2(t-1), \dots, (n-1)(t-1)) = |\mathcal{T}_{n,t}| - 1$. For example, we consider a 3-ary tree T with $rd(T) = (0, 2, 1, 0, 1, 2)$. From Table 2, we can compute $\text{Rank}(0, 2, 1, 0, 1, 2) = B_{1,10} + B_{2,10} + B_{3,7} + B_{4,4} + B_{5,3} + B_{6,2} = 0 + 728 + 55 + 0 + 3 + 2 = 788$.

Based on (12), we design the following algorithm which takes $rd(T) = (d_1, d_2, \dots, d_n)$ as the input for computing the rank of a t -ary tree T . For convenience, we assume that there exists at least one nonzero item in the sequence $d_1, d_2, \dots, d_n$. Indeed, it is unnecessary to build the accumulation table in our algorithm. An alternated way shows that using (8) and (10) instead of (7) can efficiently reduce the complexity for computing the rank.

Algorithm RANKING

Step 1. $l = \max\{i : d_i \neq 0 \text{ and } 1 \leq i \leq n\}$ and
 $s = \min\{i : d_i \neq 0 \text{ and } 1 \leq i \leq n\};$
 $B = k = (n-l)(t-1) + d_l;$

```

for  $m = 1$  to  $n - l$  do
  // Refer to (10), move up
   $B = B \cdot \frac{(k-t+1)(mt+k)}{k(m+1)}$ ;
   $k = k - (t - 1)$ ;
endfor
 $Rank = B$ ;
Step 2.  $j = l$ ;
while  $j > s$  do
   $l = \max\{i : d_i \neq 0 \text{ and } s \leq i < j\}$ 
   $m = n - j + 1$ ;
   $h = (j - l)(t - 1) + d_l - d_j$ ;
  for  $k = d_j$  to  $d_j + h - 1$  do
    // Refer to (8), move right
     $B = B \cdot \frac{(k+1)(mt+k)}{k[m(t-1)+k+1]}$ ;
  endfor
   $k = d_j + h$ ;
  for  $m = n - j + 1$  to  $n - l$  do
    // Refer to (10), move up
     $B = B \cdot \frac{(k-t+1)(mt+k)}{k(m+1)}$ ;
     $k = k - (t - 1)$ ;
  endfor
   $Rank = Rank + B$ ;
   $j = l$ ;
endwhile
Step 3. Output  $Rank$ ;

```

The algorithm RANKING calculates the rank by a sequence of nonzero terms $B_{i,(t-1)(n-i)+d_i}$ with index i from n downto 1. Step 1 first determines the largest index of nonzero term. In Step 2, we consider any two consecutive nonzero terms with indices j and l such that $j > l$. Since $d_j \leq d_l + (j - l)(t - 1)$, we have $(t - 1)(n - j) + d_j \leq (t - 1)(n - l) + d_l$. Thus, if the given RD-sequence is valid, the term with index l must appear at the upper or the upper-right of the term with index j in the t -ary accumulation table, which depends on the value of h to be zero or not. Then, we can find the next nonzero term by successively traversing the entries of the table through h steps in horizontal, and then through $j - l$ steps in vertical. We repeat this procedure until all subsequent nonzero terms are added

into the rank. Instead of table search, we obtain all nonzero terms using the calculations of (8) and (10), and therefore the correctness of algorithm directly follows. Since each of (8) and (10) can be computed in a constant time if the previous nonzero term is provided, and since the total steps traversed horizontally and vertically in the table is no more than $(n - 1)(t - 1) + (n - 1) = t(n - 1)$, we conclude the following.

Theorem 10 *Algorithm RANKING can correctly determine the rank of a t -ary tree with n internal nodes in $O(tn)$ time.*

In the following, we give a reverse procedure, called UNRANKING, that converts a positive integer N to its corresponding RD-sequence. We are given n, t and N , and look for a sequence $d_1, d_2, \dots, d_n$ such that $Rank(d_1, d_2, \dots, d_n) = N$. Initially, we assume that $d_i = 0$ for all $i = 1, \dots, n$.

Algorithm UNRANKING

```

Step 1.  $B = |\mathcal{T}_{n,t}| = \frac{1}{tn+1} \binom{tn+1}{n}$ ;
Step 2.  $i = k = 1$ ;
while  $N > 0$  do
   $i = i + 1$ ;
   $m = n - i + 1$ ;
   $k = k + (t - 1)$ ;
  while  $k > 1$  and  $B > N$  do
    // Refer to (9), move left
     $B = B \cdot \frac{(k-1)[m(t-1)+k]}{k(mt+k-1)}$ ;
     $k = k - 1$ ;
  endwhile
  if  $B \leq N$  then
     $N = N - B$ ;
     $d_i = k$ ;
  endif
  // Refer to (11), move down
   $B = B \cdot \frac{m(k+t-1)}{k(mt+k-1)}$ ;
endwhile
Step 3. Output  $(d_1, d_2, \dots, d_n)$ ;

```

The algorithm UNRANKING decomposes N into a sequence $B_{i,(t-1)(n-i)+d_i}$ for $i = 1, \dots, n$. For determining which terms are appropriate in the sequence, similar to RANKING, the algorithm performs the search in accumulation table using calculations (9) and (11) and without really building the table. Step 1 takes $O(n)$ time to pick out the upper bound of $B_{1,(t-1)(n-1)+d_1}$. In Step 2, for each round of the outer loop, it determines an entry B from a row of the table as an appropriate term, and then updates N by subtracting B from N . The choice of entry B is percolated by the inner loop which can be viewed as a search from right to left in a row of the table and such that B matches the largest possible value to accommodate the current N . Since each of (9) and (11) can be computed in a constant time and the total steps of calculation is no more than $(n-1)(t-1) + (n-1) = t(n-1)$, as a result we have

Theorem 11 *Given a positive integer N , Algorithm UNRANKING can correctly determine an RD-sequence $d_1, d_2, \dots, d_n$ of a t -ary tree such that $\text{Rank}(d_1, d_2, \dots, d_n) = N$ in $O(tn)$ time.*

4. Conclusion

In this paper, we use the RD-sequences to represent t -ary trees. Due to the conceptual simplicity of this representation, it can be applied to the recursion tree for labeling. Thus, we can investigate the structure of t -ary trees by means of the recursion tree and its concomitant tables. Furthermore, four expedient formulas are derived to reduce the complexity of ranking and unranking algorithms. This is the first time that both the ranking and unranking algorithms of t -ary trees can be run in $O(tn)$ time such that no any auxiliary tables are available. As a corollary, we obtain linear time algorithms for ranking and unranking on binary trees. Our recent work has tended towards to design efficient ranking

and unranking algorithms of t -ary trees under various parallel models. Also, we have known that many ranking and unranking algorithms have been developed for the variation of trees or other objects, such as AVL-trees [16], 2-3 trees [6], B-trees [7], etc. Using the similar approach of this paper (i.e., traversing a path in a virtual table), another direction of the future research can devote our effort to design more efficient algorithms for improving the complexity of ranking and/or unranking on any of the above mentioned objects.

References

- [1] H. Ahrabian and A. Nowzari-Dalini, Generation of t -ary trees with Ballot-sequences, *International Journal of Computer Mathematics* 80 (2003) 1243–1249.
- [2] H. Ahrabian and A. Nowzari-Dalini, Adaptive generation of t -ary trees in parallel, *The Electronic International Journal Advanced Modeling and Optimization* 8 (2006) 19–28.
- [3] H. Ahrabian, A. Nowzari-Dalini, and E. Salehi, Gray code algorithm for listing k -ary trees, *Studies in Informatics and Control* 13 (2004) 243–251.
- [4] S. G. Akl and I. Stojmenović, Generating t -ary trees in parallel, *Nordic Journal of Computing* 3 (1996) 63–71.
- [5] M. C. Er, Efficient generation of k -ary trees in natural order, *The Computer Journal* 35 (1992) 306–308.
- [6] U. Gupta, D. T. Lee, and C. K. Wong, Ranking and unranking of 2-3 trees, *SIAM Journal on Computing* 11 (1982) 582–590.
- [7] U. Gupta, D. T. Lee, and C. K. Wong, Ranking and unranking of B-trees, *Journal of Algorithms* 4 (1983) 51–60.
- [8] D. A. Klarner, Correspondences between plane trees and binary sequences, *Journal of Combinatorial Theory* 9 (1970) pp. 401–411.
- [9] D. E. Knuth, *The Art of Computer Programming. Vol. 1: Fundamental Algorithms*, Addison-Wesley, Reading, MA, 1968.

- [10] Z. Kokosiński, On parallel generation of t -ary trees in an associative model, in: *Proc. of 4th International Conference on Parallel Processing and Applied Mathematics* (PPAM 2001), LNCS 2328, Springer, (2002) pp. 228–235.
- [11] Z. Kokosiński, A parallel dynamic programming algorithm for unranking t -ary trees, in: *Proc. of 5th International Conference on Parallel Processing and Applied Mathematics* (PPAM 2003), LNCS 3019, Springer, (2004) pp. 255–260.
- [12] J. F. Korsh, Loopless generation of k -ary tree sequences, *Information Processing Letters* 52 (1994) 243–247.
- [13] J. F. Korsh, Generating t -ary trees in linked representation, *The Computer Journal* 48 (2005) 488–497.
- [14] J. F. Korsh and P. LaFollette, Loopless generation of Gray code for k -ary trees, *Information Processing Letters* 70 (1999) 7–11.
- [15] J. F. Korsh and S. Lipschutz, Shifts and loopless generation of k -ary trees, *Information Processing Letters* 65 (1998) 235–240.
- [16] L. Li, Ranking and unranking of AVL-trees, *SIAM Journal on Computing* 15 (1986) 1025–1035.
- [17] J. M. Lucas, D. Roelants van Baronaigien, and F. Ruskey, On rotations and the generation of binary trees, *Journal of Algorithms* 15 (1993) 343–366.
- [18] E. Mäkinen, Left distance binary tree representations, *BIT* 27 (1987) 163–169.
- [19] E. Mäkinen, Generating random binary trees – A survey, *Information Sciences* 115 (1999) 123–136.
- [20] H. W. Martin and B. J. Orr, A random binary tree generator, in: *Proc. of the 17th conference on ACM Annual Computer Science Conference*, Louisville, Kentucky, February 21–23, 1989, pp. 33–38.
- [21] D. Roelants van Baronaigien, A loopless Gray-code algorithm for listing k -ary trees, *Journal of Algorithms* 35 (2000) 100–107.
- [22] D. Roelants van Baronaigien and F. Ruskey, Generating t -ary trees in A-order, *Information Processing Letters* 27 (1988) 205–213.
- [23] F. Ruskey, Generating t -ary trees lexicographically, *SIAM Journal on Computing* 7 (1978) 424–439.
- [24] N. J. A. Sloane, Sequence [A009766](#) in "The On-Line Encyclopedia of Integer Sequences."
- [25] A. E. Trojanowaki, Ranking and listing algorithms for k -ary trees, *SIAM Journal on Computing* 7 (1978) 492–509.
- [26] V. Vajnovszki, C. Phillips, Optimal parallel algorithm for generating k -ary trees, in: *Proc. of the 12th International Conference on Computers and Their Applications*, Tempe, Arizona, (ed. M. C. Woodfill), March 13–15, 1997, pp. 201–204.
- [27] V. Vajnovszki and C. Phillips, Systolic generation of k -ary trees, *Parallel Processing Letters* 9 (1999), 93–101.
- [28] L. Xiang, K. Ushijima, and S. G. Akl, Generating regular k -ary trees efficiently, *The Computer Journal* 43 (2000) 290–300.
- [29] L. Xiang, K. Ushijima, and Y. Asahiro, Coding k -ary trees for efficient loopless generation in lexicographic order, in: *Proc. of International conference on Information Technology: Coding and Computing* (ITCC'02), IEEE Computer Society Press, Las Vegas, April 8–10, 2002, pp. 396–401.
- [30] L. Xiang, K. Ushijima, and C. Tang, Efficient loopless generation of Gray codes for k -ary trees, *Information Processing Letters* 76 (2000) 169–174.
- [31] L. Xiang, K. Ushijima, and C. Tang, On generating k -ary trees in computer representation, *Information Processing Letters* 77 (2001) 231–238.
- [32] S. Zaks, Lexicographic generation of ordered trees, *Theoretical Computer Science* 10 (1980) 63–82.
- [33] S. Zaks, Generation and ranking of k -ary trees, *Information Processing Letters* 14 (1982) 44–48.
- [34] D. Zerling, Generating binary trees using rotations, *Journal of the ACM* 32 (1985) 694–701.

Appendix

A. The proof of Theorem 2

The proof is by induction on m . For $m = 1$ (i.e., $i = n$), it is easy to check $A_{n,k} = \frac{(t+k)}{(t+k+1)(t+k)} \binom{t+k+1}{1} = 1$ for $0 \leq k \leq (t-1)(n-1)$. Suppose that the formula (4) holds for $m = l < n$, i.e., $A_{n-l+1,(t-1)(l-1)+k} = \frac{l(t+k)}{(lt+k+1)(lt+k)} \binom{lt+k+1}{l} = \frac{t+k}{lt+k} \binom{lt+k}{l-1}$, where $0 \leq k \leq (t-1)(n-l)$. We now consider $m = l+1$ (i.e., $i = n-l$) and let k be any positive integer which is no more than $(t-1)(n-l-1)$. By (3), we have

$$\begin{aligned}
A_{n-l,(t-1)l+k} &= \sum_{j=(t-1)(l-1)}^{(t-1)l+k} A_{n-l+1,j} \\
&= A_{n-l+1,(t-1)(l-1)} + A_{n-l+1,(t-1)(l-1)+1} + A_{n-l+1,(t-1)(l-1)+2} + \cdots + A_{n-l+1,(t-1)l+k} \\
&= \frac{1}{l} \binom{lt}{l-1} + \frac{t+1}{lt+1} \binom{lt+1}{l-1} + \frac{t+2}{lt+2} \binom{lt+2}{l-1} + \cdots + \frac{t+(t-1+k)}{lt+(t-1+k)} \binom{lt+(t-1+k)}{l-1} \\
&= \left[\frac{1}{l} \cdot \frac{lt-l+2}{lt+1} + \frac{t+1}{lt+1} \right] \binom{lt+1}{l-1} + \frac{t+2}{lt+2} \binom{lt+2}{l-1} + \cdots + \frac{t+(t-1+k)}{lt+(t-1+k)} \binom{lt+(t-1+k)}{l-1} \\
&= \frac{2}{l} \binom{lt+1}{l-1} + \frac{t+2}{lt+2} \binom{lt+2}{l-1} + \cdots + \frac{t+(t-1+k)}{lt+(t-1+k)} \binom{lt+(t-1+k)}{l-1} \\
&= \left[\frac{2}{l} \cdot \frac{lt-l+3}{lt+2} + \frac{t+2}{lt+2} \right] \binom{lt+2}{l-1} + \frac{t+3}{lt+3} \binom{lt+3}{l-1} + \cdots + \frac{t+(t-1+k)}{lt+(t-1+k)} \binom{lt+(t-1+k)}{l-1} \\
&= \frac{3}{l} \binom{lt+2}{l-1} + \frac{t+3}{lt+3} \binom{lt+3}{l-1} + \cdots + \frac{t+(t-1+k)}{lt+(t-1+k)} \binom{lt+(t-1+k)}{l-1} \\
&= \cdots \\
&= \frac{t+k}{l} \binom{lt+(t-1+k)}{l-1} \\
&= \frac{t+k}{(l+1)t+k} \binom{(l+1)t+k}{l}
\end{aligned}$$

□

B. The proof of Theorem 5

Obviously, if $k = 0$ then $B_{i,(t-1)(n-i)} = 0$. We now consider $k \geq 1$ and using (6) and (4) for the following computation:

$$\begin{aligned}
B_{i,(t-1)(n-i)+k} &= \sum_{j=0}^{k-1} A_{i,(t-1)(n-i)+j} \\
&= A_{i,(t-1)(n-i)} + A_{i,(t-1)(n-i)+1} + A_{i,(t-1)(n-i)+2} + \cdots + A_{i,(t-1)(n-i)+(k-1)} \\
&= \frac{mt}{(mt+1)mt} \binom{mt+1}{m} + \frac{m(t+1)}{(mt+2)(mt+1)} \binom{mt+2}{m} + \frac{m(t+2)}{(mt+3)(mt+2)} \binom{mt+3}{m} + \cdots \\
&\quad + \frac{m(t+k-1)}{(mt+k)(mt+k-1)} \binom{mt+k}{m} \\
&= \frac{2}{mt+2} \binom{mt+2}{m} + \frac{m(t+2)}{(mt+3)(mt+2)} \binom{mt+3}{m} + \cdots + \frac{m(t+k-1)}{(mt+k)(mt+k-1)} \binom{mt+k}{m} \\
&= \frac{3}{mt+3} \binom{mt+3}{m} + \frac{m(t+3)}{(mt+4)(mt+3)} \binom{mt+4}{m} + \cdots + \frac{m(t+k-1)}{(mt+k)(mt+k-1)} \binom{mt+k}{m} \\
&= \cdots \\
&= \frac{k}{mt+k} \binom{mt+k}{m}
\end{aligned}$$

□